European Journal of Operational Research 326 (2025) 220–233

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Discrete Optimization

On the integration of reinforcement learning and simulated annealing for the parallel batch scheduling problem with setups

Gustavo Alencar Rolim [a,*], Caio Paziani Tomazella [b,c], Marcelo Seido Nagano [a]

[a] Department of Production Engineering, Sao Carlos School of Engineering, University of Sao Paulo, Sao Carlos, Brazil
[b] Department of Applied Mathematics and Statistics, Institute of Mathematical and Computer Sciences, University of Sao Paulo, Sao Carlos, Brazil
[c] Department of Production Engineering, Federal University of Sao Carlos, Sao Carlos, Brazil

ARTICLE INFO

Keywords:
Metaheuristics
Reinforcement learning
Simulated annealing
Adaptive large neighborhood search
Stochastic local search

ABSTRACT

Motivated by semiconductor applications, where wafer lots are grouped into families and processed on batch machines, this paper addresses a generalized unrelated parallel-batch scheduling problem. The goal is to minimize total completion time (flow time) while considering family- and machine-dependent setup times. We propose a mixed-integer programming formulation, establish a necessary condition for optimal schedules, and develop a polynomial-time heuristic for batching and sequencing. We also evaluate Q-Learning, a model-free reinforcement learning algorithm, for neighborhood selection within two Simulated Annealing-based metaheuristics: Stochastic Local Search (SLS) and Adaptive Large Neighborhood Search (ALNS). Results show that SLS and ALNS achieve better solutions and faster convergence compared to existing approaches. Finally, we conclude that while Q-Learning has the potential to improve solution quality in certain cases, it also increases the complexity of the algorithms, making them harder to configure and scale.

1. Introduction

In recent years, the field of scheduling has seen a growing interest in problems involving batch determination decisions, driven by the increasing pressure to reduce production costs and satisfy customer demands. Batch processing, as opposed to individual job processing, offers considerable efficiency advantages, making it critical for large-scale manufacturing (Gahm et al., 2021). Scheduling batch processing machines (BPMs) may include special considerations. For example, setting up machines for jobs with differing characteristics often requires tasks like tool changes, material positioning, and cleaning. Family scheduling, where jobs are grouped based on similarities, addresses this challenge, however, setups are still necessary when switching between families (Allahverdi et al., 1999).

In addition to family grouping and setup considerations, BPMs are subject to a capacity limit. In the semiconductor industry, the volume of a job is a critical parameter that could represent either the physical size or other relevant job characteristic. The concept of job volume extends beyond physical dimensions. It can also encompass attributes like energy absorption when there is a constraint on the total energy absorbed within a batch (Dobson & Nambimadom, 2001). These concepts are not confined to a specific segment, instead, they find applications in various sectors such as automotive parts manufacturing (Gokhale & Mathirajan, 2011), healthcare (Ozturk et al., 2012), brewing (Sáenz-Alanís et al., 2016), and electricity (Yang et al., 2022).

The emphasis on problems involving BPMs has evolved considerably since the pioneering paper by Ikura and Gimple (1986). However, it is noteworthy that the vast majority of existing literature has been predominantly centered on the single and identical parallel machines environments. This focus is evident in the recent survey conducted by Fowler and Mönch (2022). Consequently, research on unrelated BPMs remains relatively scarce. In this paper, we are concerned with the scheduling of unrelated BPMs in which jobs belong to incompatible families and have arbitrary sizes, that is, batches are exclusively formed by jobs belonging to the same family as long as they respect a machine capacity constraint. Following a parallel batching model (p-batch), the processing time of a batch is determined by the maximum processing time of the jobs grouped within the same batch and may change according to the machine in which the batch is scheduled. Furthermore, we take into account the presence of setups when transitioning between batches from different families with the objective of minimizing the total completion times of jobs. Using the three-field classification scheme by Graham et al. (1979) and the notations by Rolim and Nagano (2020) we refer to this problem as $R|\mathbf{B}, f, \mathrm{ST}_{\mathrm{sd,b}}|\sum C_j$.

Recently, there has been a rising trend in adopting Reinforcement Learning (RL) in metaheuristics to solve combinatorial optimization

* Corresponding author.
E-mail addresses: gustavo.rolim@usp.br (G.A. Rolim), caio.tomazella@usp.br (C.P. Tomazella), drnagano@usp.br (M.S. Nagano).

https://doi.org/10.1016/j.ejor.2025.04.042
Received 29 April 2024; Accepted 23 April 2025
Available online 2 May 2025
0377-2217/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

problems. Some interesting results have been reported by Alicastro et al. (2021), where Q-Learning, a model-free RL algorithm, is used to dynamically select neighborhood structures during the search process of an iterated local search. Similarly, Karimi-Mamaghan et al. (2023) applied Q-Learning to determine the destruction degree in an iterated greedy (IG) algorithm. Q-Learning has also been used on both cases, as reported by Chen and Wang (2024). Our study has therefore two main perspectives: firstly, proposing well-designed methods to address the unrelated parallel batch scheduling problem with family setups. Secondly, assessing the advantages of integrating Q-Learning into our solution approaches. Particularly, we list the following contributions:

- We conceptualize the problem, present a mixed-integer programming (MIP) formulation, and show a necessary but not sufficient condition for optimal schedules.
- We present a constructive heuristic, define several neighborhood structures, an Adaptive Large Neighborhood Search (ALNS), and a Stochastic Local Search (SLS).
- We propose a framework to integrate Q-Learning to the neighborhood selection process of both ALNS and SLS.
- We analyze the impact and discuss the challenges of incorporating Q-Learning for neighborhood selection. This is a trending topic in the field of metaheuristics as attested by numerous survey papers (Karimi-Mamaghan et al., 2022; Kayhan & Yildiz, 2023; Talbi, 2021). We also quantitatively compare the improvements of Q-Learning against the classical adaptive layer of ALNS.
- As increasingly complex and overly hybridized heuristics emerge daily, we adopt a *less-is-more* approach, gradually integrating SA-based metaheuristics and Q-Learning to evaluate whether designing a more complex heuristic is justified (Brimberg et al., 2023; Mladenović et al., 2022).

The rest of this paper is organized as follows. Section 2 provides a concise background on related scheduling problems. Section 3 describes the problem statement. Section 4 introduces a constructive procedure. Sections 5 and 6 discuss the ALNS and SLS procedures, respectively. Section 7 outlines the integration framework of Q-Learning and the proposed metaheuristics. Section 8 details the experimental setup and numerical results. Finally, Section 9 discusses concluding remarks and future research directions.

2. Background

We present a consolidation of the existing literature on scheduling involving parallel BPMs. More specifically, we are concerned with problems involving the minimization of total completion time. For a broader perspective of the field we recommend the surveys by Fowler and Mönch (2022), Mathirajan and Sivakumar (2006), and Potts and Kovalyov (2000). For online problems, we refer to Tian et al. (2014).

2.1. Compatible families

To the best of our knowledge, Chandru et al. (1993) was the first to address the problem of scheduling identical parallel BPMs to minimize total completion time. In their model, all jobs belong to the same family and have equal capacity requirements. They reported that a greedy ratio heuristic presented better results than a full-batch shortest processing time heuristic. Later, Hochbaum and Landy (1997) improved previous results by showing an exact pseudopolynomial time algorithm and a polynomial approximation scheme with a performance guarantee of $(1+\sqrt{2})/2 \approx 1.2$ times the optimal solution. Cheng et al. (2012) showed a polynomial-time algorithm that runs in $\mathcal{O}(n \log n)$ when jobs have different sizes and processing times. Li et al. (2021) showed that the worst-case ratio of the first fit increasing heuristic adapted from the bin-packing problem is no more than $(6+\sqrt{2})/4 \approx 1.8536$ for scheduling parallel identical BPMs with equal processing times and different job sizes.

Jia et al. (2018) studied the minimization of the sum of weighted completion times on identical BPMs when jobs have identical processing times and showed an algorithm based on ant colony optimization (ACO). Arroyo et al. (2019) proposed an IG algorithm for the problem with release dates and unequal machine capacities. Their IG demonstrated a performance superior to several other related algorithms from the literature. Ozturk (2020) proposed a truncated column generation procedure for instances with up to 50 jobs. Zhang et al. (2020) showed an ACO algorithm for a similar problem, however, with the assumption that job sizes may exceed the capacity of some of the available machines. In a follow up paper, Zhang et al. (2023) considered unrelated BPMs, included the restriction of unequal ready times, and compared the results of a modified elite ant system with local search (MEASL) with other algorithms from the literature by setting a limit on the number of iterations.

2.2. Incompatible families

Uzsoy (1995) studied the problem of scheduling parallel BPMs with incompatible families for the minimization of makespan, total weighted completion time, and maximum lateness criteria. He proposed a solution approach based on the greedy weighted completion time (GWCT) heuristic that requires $\mathcal{O}(n \log n)$ time and a list scheduling algorithm. Koh et al. (2004) extended this problem to the case in which jobs have different sizes and should respect a machine capacity. They study different performance criteria such as makespan, total completion time, and total weighted completion, and presented genetic algorithms (GAs) with different decoding schemes to tackle the problem.

Klemmt et al. (2011) studied a challenging scheduling problem involving unrelated BPMs with incompatible job families and release times. They modeled the problem from both the deterministic and stochastic perspectives aiming at minimizing the makespan, total completion time, and total tardiness. A GA and a MIP formulation based on time decomposition were proposed.

Cakici et al. (2013) modeled a supply chain problem in which transportation units can be viewed as identical BPMs that group orders that should be processed together. Jobs become available for pickup at different times and are delivered to customers by vehicles with a finite capacity. The authors showed different versions of a variable neighborhood search (VNS) scheme for total weighted completion time minimization. Ham et al. (2017) proposed a constraint-programming (CP) model and used the instances from Cakici et al. (2013) as a benchmark to measure its efficiency. They showed that the CP model performed better than the existing MIP formulations. More recently, Jiang et al. (2022) also developed a CP model and some IG algorithms for the case of unrelated BPMs with unequal release dates. Shahvari et al. (2022) propose an exact branch-and-price algorithm for a complex problem involving unrelated BPMs to minimize a linear combination of total weighted completion times and tardiness.

Despite the advances in BPMs scheduling, there are only few studies dealing with unrelated machines. This is important in semiconductor manufacturing since machines at diffusion workstations may have different processing times requirements. Moreover, wafer lots are often categorized in product families with setups incurred when transitioning between families (Kim et al., 2010). With that said, we propose several heuristic procedures to deal with the problem of minimizing total completion times.

3. Problem statement

We schedule a set of $\mathcal{J} = \{1, \dots, n\}$ jobs, in which each job belongs to precisely one out of f possible families. The set of incompatible job families is denoted by $\mathcal{F} = \{1, \dots, f\}$ in such a way that every family $k \in \mathcal{F}$ consists of a subset $\mathcal{J}_k \subseteq \mathcal{J}$. More specifically, we assume that

$\cup_{k=1}^{f} \mathcal{J}_k = \mathcal{J}$ and $\mathcal{J}_k \cap \mathcal{J}_l = \emptyset$ for every two different families $k, l \in \mathcal{F}$ with $k \neq l$. Each job $j \in \mathcal{J}$ needs to be processed in exactly one out of a set of $\mathcal{M} = \{1, \ldots, m\}$ unrelated BPMs. All jobs are available at time zero and no preemption is allowed. Additionally, each job $j \in \mathcal{J}$ has an associated processing time $p_{ij} \in \mathcal{P}$ on machine $i \in \mathcal{M}$ and a size $v_j \in \mathcal{V}$. The BPMs can process several jobs from the same family simultaneously as long as the maximum capacity $v_{\max}$ is respected. In this study, we assume $0 \leq v_j \leq v_{\max}$.

A feasible batch is therefore defined as a subset of jobs $\mathcal{B} \subseteq \mathcal{J}$ that can be executed simultaneously, if and only if, all jobs belong to the same family, i.e. $\mathcal{B} \subseteq \mathcal{J}_k$ for any given $k \in \mathcal{F}$, and if the total batch size do not exceed the machine capacity, i.e. $\sum_{j \in \mathcal{B}} v_j \leq v_{\max}$. The processing time of a batch assigned to machine $i \in \mathcal{M}$ is defined as $\max_{j \in \mathcal{B}} p_{ij}$ in such a way that all jobs in the same batch start and finish at the same time. Furthermore, a setup time $s_{ikl} \in \mathcal{S}$ is incurred when processing a batch from family l after a batch from family k in machine i. We also consider a setup s_{i0k} (machine preparation) if a batch from family k is the first to be processed in machine i. A solution, or schedule, consists of the allocation of each job to a feasible batch, the assignment of each batch to a machine, and the sequencing of these batches on each machine. If we denote the completion time of job j by C_j then our goal is to find a schedule Π that minimizes the total completion time $\sum_{j \in \mathcal{J}} C_j$. Fig. 1 shows a conceptual model of the problem. Job and batch colors indicate their respective families.

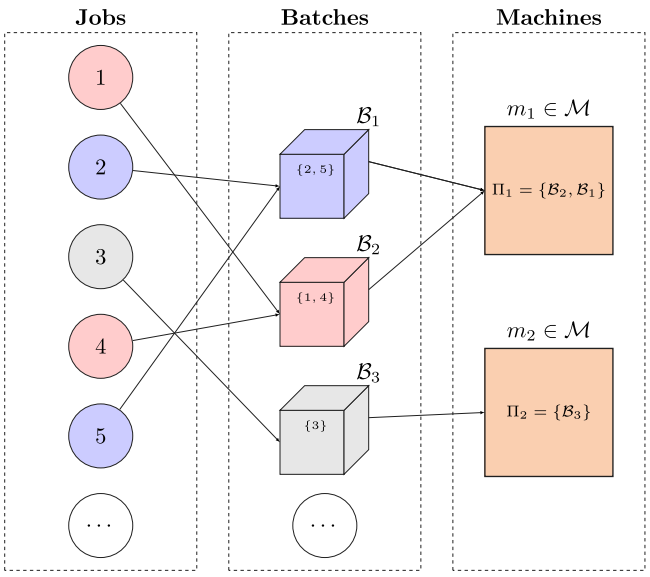


Fig. 1. Conceptual model of the problem under study.

3.1. Mathematical formulation

We now present a MIP formulation for the problem under study. Our formulation is based on assignment and positional variables similarly to the one discussed in Unlu and Mason (2010). The notations used in the formulation, and throughout the rest of the paper, are described next.

Sets, indices, and parameters

$j \in \mathcal{J}$	Job index.
$k, l \in \mathcal{F}$	Family indices.
b	Batch index.
$i \in \mathcal{M}$	Machine index.
$p_{ij} \in \mathcal{P}$	Processing time of job j on machine i.
$v_j \in \mathcal{V}$	Size of job j.
$v_{\max}$	Machine capacity.
$s_{ikl} \in \mathcal{S}$	Setup of machine i between batches from families k and l.
M	Sufficiently large number.
k_j	Family of job j

Decision variables

X_{ijb}	$\begin{cases} 1, & \text{if job } j \text{ is assigned to batch } b \text{ on machine } i. \\ 0, & \text{otherwise.} \end{cases}$
Y_{ibk}	$\begin{cases} 1, & \text{if family } k \text{ is assigned to batch } b \text{ on machine } i. \\ 0, & \text{otherwise.} \end{cases}$
P_{ib}	Processing time of batch b on machine i.
C_{ib}	Completion time of batch b on machine i.
C_j	Completion time of job j.

$$\min \sum_{j=1}^{n} C_j \tag{1}$$

s.t.

$$\sum_{i=1}^{m} \sum_{b=1}^{n} X_{ijb} = 1 \qquad j \in \mathcal{J} \tag{2}$$

$$X_{ijb} \leq Y_{ibk_j} \qquad i \in \mathcal{M}; j, b \in \mathcal{J} \tag{3}$$

$$\sum_{k=1}^{f} Y_{ibk} \leq 1 \qquad i \in \mathcal{M}; b \in \mathcal{J} \tag{4}$$

$$\sum_{j=1}^{n} X_{ij(b-1)} \geq \sum_{k=1}^{f} Y_{ibk} \qquad i \in \mathcal{M}; b \in \mathcal{J}, b \geq 2 \tag{5}$$

$$\sum_{j=1}^{n} v_j \cdot X_{ijb} \leq v_{\max} \qquad i \in \mathcal{M}; b \in \mathcal{J} \tag{6}$$

$$p_{ij} \cdot X_{ijb} \leq P_{ib} \qquad i \in \mathcal{M}; j, b \in \mathcal{J} \tag{7}$$

$$C_{i1} \geq s_{i0k} + P_{i1} - M \cdot (1 - Y_{i1k}) \qquad i \in \mathcal{M}; k \in \mathcal{F} \tag{8}$$

$$C_{ib} \geq C_{i(b-1)} + [s_{ikl} - M \cdot (1 - Y_{i(b-1)k})] + [P_{ib} - M \cdot (1 - Y_{ibl})] \quad i \in \mathcal{M}; b \in \mathcal{J}, b \geq 2; k, l \in \mathcal{F} \tag{9}$$

$$C_j \geq C_{ib} - M \cdot (1 - X_{ijb}) \qquad i \in \mathcal{M}; j, b \in \mathcal{J} \tag{10}$$

$$X_{ijb} \in \{0, 1\} \qquad i \in \mathcal{M}; j, b \in \mathcal{J} \tag{11}$$

$$Y_{ibk} \in \{0, 1\} \qquad i \in \mathcal{M}; b \in \mathcal{J}; k \in \mathcal{F} \tag{12}$$

$$P_{ib}, C_{ib} \geq 0 \qquad i \in \mathcal{M}; b \in \mathcal{J} \tag{13}$$

$$C_j \geq 0 \qquad j \in \mathcal{J} \tag{14}$$

Objective function (1) minimizes the total completion time of the schedule. Constraints (2) guarantee that each job is assigned to exactly one batch. In Constraints (3), a family is assigned to a batch if at least one job belonging to this family is assigned to it as well, while Constraints (4) set a limit of at most one family assigned per batch. Constraints (5) are symmetry breaking constraints, that, for any given machine i, do not allow jobs to be assigned to batch b if no other jobs were assigned to batch $b-1$ yet. This is done to avoid the scheduling of empty batches between two existing batches, which would lead to the occurrence of multiple solutions representing the same schedule. Constraints (6) restrict the combination of jobs assigned to a batch to the maximum machine capacity, and Constraints (7) define the processing time of a batch as the longest of the processing times of its jobs. Constraints (8)–(9) calculate the completion time of the first batch on each machine and the remaining batches, respectively. The completion time of each individual job is defined by Constraints (10) as the completion time of the batch the job is assigned to. Lastly, the variable domains are defined with Constraints (11)–(14).

3.2. Optimal schedules and a dominance relationship

In the past few decades, important theoretical results were achieved for closely related problems involving the minimization of the total

completion time. Both single and parallel-machine cases can benefit from the well-known Smith's rule, or shortest processing time (SPT) first. For instance, problems like $1||\sum C_j$ and $P||\sum C_j$ may benefit from SPT. The first problem is solvable in polynomial time through the direct application of SPT (Kramer et al., 2021; Smith, 1956). Since our problem has additional considerations such as job families, setups, and limited machine capacity, we conducted a preliminary study on optimal schedules to verify the existence of dominance properties. Fig. 2 depicts a sensitivity analysis of the number of job families when $n = 10$ and $m = 2$. More specifically, Figs. 2(a), 2(b), and 2(c) represent instances with $f = 2$, $f = 4$, and $f = 6$, respectively. Details of the data of each problem are available in Section 8.

It is evident that a necessary but not sufficient condition for an optimal schedule can be obtained as an extension of the weighted shortest processing time (WSPT) rule (Crauwels et al., 1998; Dunstall et al., 2000; Mason & Anderson, 1991; Monma & Potts, 1989). As an implication of this condition, the following relationship holds:

Proposition 1. *Consider a schedule Π and let Π_i denote a sequence of jobs in machine i. Now, let $\pi = \{\mathcal{B}_1, \ldots, \mathcal{B}_{|\pi|}\}$ be a sub-sequence of $|\pi|$ feasible batches from the same family k such that $\mathcal{B}_b \in \pi$ implies that $\mathcal{B}_b \subseteq \mathcal{J}_k, k \in \mathcal{F}$. If P_{ib} is the processing time of batch b in machine i and $|\mathcal{B}_b|$ the number of jobs in batch $\mathcal{B}_b$, then it is optimal to sequence consecutive batches from the same family in a batch weighted shortest processing time (BWSPT) order, i.e.*

$$\frac{P_{i[1]}}{|\mathcal{B}_{[1]}|} \leq \frac{P_{i[2]}}{|\mathcal{B}_{[2]}|} \leq \cdots \leq \frac{P_{i[|\pi|-1]}}{|\mathcal{B}_{[|\pi|-1]}|} \leq \frac{P_{i[|\pi|]}}{|\mathcal{B}_{[|\pi|]}|}$$

where $[x]$ indicates the batch in the xth position of the sub-sequence.

Proof. Suppose that batch b completes its processing on time C_{ib}. Since all the jobs in the batch are assumed to complete at the same time, the contribution of this batch to the objective function will be $|\mathcal{B}_b| \cdot C_{ib}$. If we interpret the batch as an aggregate job with processing time P_{ib} and weight $|\mathcal{B}_b|$, the sub-sequences of batches from the same family in each machine can be viewed as the minimization of the weighted sum of completion times in a single machine with $|\pi|$ jobs corresponding to $|\pi|$ batches. It is well known that SWPT rule is optimal for the equivalent problem, therefore a simple proof by contradiction can be obtained (Pinedo, 2022). □

It is evident that, as a consequence of Proposition 1, all sub-sequences of batches from the same family should be arranged according to the BWSPT rule. This information is particularly useful for a post-optimization procedure of any dominated feasible schedule. However, it is worth noting that producing high-quality solutions remains challenging. Exact approaches still prohibitive in terms of time and memory requirements, specially when the number of jobs and machines increases as evidenced in the recent paper by Yang et al. (2022). They also show that the single-machine case is strongly $\mathcal{NP}$-hard and, decomposing the problem into batching, which can be solved running a bin-packing routine, and scheduling sub-problems may yield poor solutions. Given these circumstances, we propose heuristic procedures based on several neighborhood structures that simultaneously consider both batching and scheduling aspects.

4. A constructive heuristic for scheduling parallel BPMs

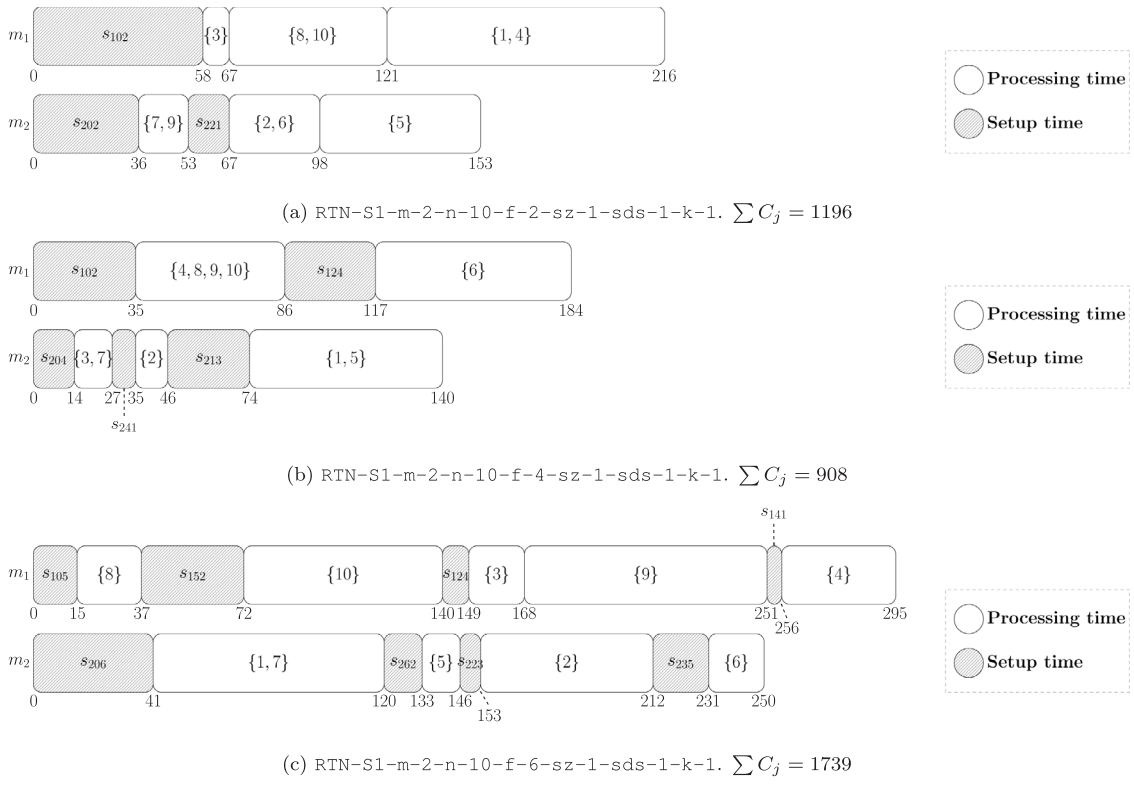
A common strategy for generating initial solutions involves job assignment to machines followed by the formation of feasible batches using bin-packing heuristics. Li et al. (2013) investigated constructive heuristics for scheduling unrelated BPMs, with their most effective heuristic comprising two phases. In the initial phase, jobs are assigned to machines based on a completion time criterion. The subsequent phase utilizes a best-fit (BF) or first-fit (FF) rule to group jobs into batches. In a later study, Arroyo and Leung (2017) compared the heuristics derived from Li et al. (2013) with an alternative method that presented better results. This alternative method, called H3, starts by creating a new batch on the machine with the lowest cost increase, followed by evaluating the insertion of jobs into existing batches or the creation of new batches at the end of each machine sequence.

A logical starting point for generating an initial solution is to rely on the H3 heuristic. However, its direct application could overlook characteristics such as setups and family constraints. The basic idea of our approach is to consider that a candidate job can be inserted into any compatible batch, that is, respecting the family and capacity constraints. Additionally, it evaluates whether creating a new singleton batch, i.e. a batch containing only one job, and inserting it in all possible positions leads to a smaller cost increase.

Algorithm 1: Constructive procedure

```
   Input  : F, J, M, P, S, V, v_max
   Output: A schedule Π, ΣC_j
1  Π_i ← ∅   i ∈ M;
2  B^new ← ∅;
3  Sort J according to min_{i∈M}{p_ij}, j ∈ J;
   /* Find the job (j*) and the movement that produces
      the minimum cost increase                          */
4  while J ≠ ∅ do
5      for k ∈ F do
6          Let j' be the highest priority job of family k;
7          B^new ← B^new ∪ {j'};
8          for i ∈ M do
9              for x = 1 to |Π_i| + 1 do
10                 Compute the cost of Π_i ∪ {B^new};
11                 if v_j' + Σ_{j∈B_[x]} v_j ≤ v_max ∧ F(j') = F(B_[x]) then
12                     Compute the cost of B_[x] ∪ {j'};
13                 end
14             end
15         end
16         B^new ← B^new \ {j'};
17     end
       /* Insert j* in a new batch or create a new one    */
18     if Z^job < Z^batch then
19         B_[x*] ← B_[x*] ∪ {j*}
20     else
21         B* ← B* ∪ {j*};
22         Π_i* ← Π_i* ∪ {B*}
23     end
24     J ← J \ {j*};
25 end
```

Algorithm 1 outlines the details of our constructive approach, which starts by sorting each job $j \in \mathcal{J}$ according to $\min_{i\in\mathcal{M}}\{p_{ij}\}$. Sorting jobs according to the SPT rule is known to enhance initial solutions for minimizing total completion time. As noted by Arroyo et al. (2019), their experimental study found that, among several initialization rules, SPT performed better. Then, it proceeds by evaluating two types of moves: the insertion of a new singleton batch $\mathcal{B}^{\text{new}}$ containing the candidate job j' or the insertion of j' into an existing batch in position $[x]$. We note that the candidate jobs are the highest priority jobs from each family (see line 5). The procedure computes the costs of both batch and job insertions for each machine sequence $\Pi_i, i \in \mathcal{M}$. Of course, j' is only inserted in batches from the same family, i.e. $F(j') = F(\mathcal{B}_{[x^*]})$, if the capacity constraint is respected (see line 11). If the cost Z^{job} of inserting the best candidate job j^* in an existing batch is less than the cost Z^{batch} of creating a new singleton batch, then j^* is inserted in the best possible batch $\mathcal{B}_{[x^*]}$ of machine i^* and position $[x^*]$. Otherwise, a new batch $\mathcal{B}^*$ is created and inserted in the best possible position of sequence Π_{i^*}. It is easy to see that the time complexity of the constructive heuristic is bounded by $\mathcal{O}(f \cdot m \cdot n^2)$. Moreover, if evaluating



(a) RTN-S1-m-2-n-10-f-2-sz-1-sds-1-k-1. $\sum C_j = 1196$

(b) RTN-S1-m-2-n-10-f-4-sz-1-sds-1-k-1. $\sum C_j = 908$

(c) RTN-S1-m-2-n-10-f-6-sz-1-sds-1-k-1. $\sum C_j = 1739$

Fig. 2. Optimal schedules of small instances: $m = 2$, $n = 10$ and $f \in \{2, 4, 6\}$.

the cost increase of each move is done in $\mathcal{O}(n)$ and two moves are evaluated, the overall complexity can be estimated in $\mathcal{O}(2 \cdot f \cdot m \cdot n^3)$.

5. Stochastic local search

Stochastic Local Search (SLS) usually refers to a vast spectrum of methodologies ranging from simple iterative improvement algorithms to evolutionary approaches (Hoos & Stützle, 2015). While there is a trend of hybridized methods in metaheuristics research, specially for solving parallel-machines scheduling problems, they are also becoming more complex, computationally expensive, and harder to understand and apply (Ulaga et al., 2022). In contrast with this trend, we add as few components as possible to the SLS while trying to produce the best possible outcome. We adopt a framework based on the SLS algorithm by Santos et al. (2016). The algorithm modifies a solution based on the alternation between two policies: *random perturbation* and *intensification*. The former prevents the algorithm from being trapped into local optima while the latter explores all subset of neighboring solutions and executes the movement that produces the best improvement.

5.1. Neighborhood structures

A proper definition of neighborhood structures is the basis of any SLS algorithm. These neighborhoods should incorporate the characteristics of the problem to avoid a biased search. We developed six neighborhood structures for a *random perturbation* policy ($\mathcal{N}_1$ to $\mathcal{N}_6$) and four for an *intensification* policy ($\mathcal{N}_7$ to $\mathcal{N}_{10}$):

- *Batch split* ($\mathcal{N}_1$): A random batch $\mathcal{B}$ is selected from a sequence $\Pi_i, i \in \mathcal{M}$. A cut point $c \in [2, |\mathcal{B}| - 1]$ is also randomly chosen. After, two new batches replace $\mathcal{B}$: $\mathcal{B}'$ with the first c jobs and $\mathcal{B}''$ with the remaining ones (Queiroga et al., 2021).
- *Batch shift* ($\mathcal{N}_2$): A new solution is generated by shifting an existing batch from its original position to a new random position on the same machine.
- *Batch move* ($\mathcal{N}_3$): A new solution is generated by moving a random batch from machine i' to a random position of machine $i'', i' \neq i''$.
- *Batch-machine swap* ($\mathcal{N}_4$): Two random batches $\mathcal{B}' \in \Pi_{i'}$ and $\mathcal{B}'' \in \Pi_{i''}$ are selected, $i' \neq i''$. After, $\mathcal{B}'$ is removed from $\Pi_{i'}$ and inserted on any position of $\Pi_{i''}$. Conversely, $\mathcal{B}''$ is removed from $\Pi_{i''}$ and inserted in any position of $\Pi_{i'}$.
- *Direct batch swap* ($\mathcal{N}_5$): Similarly to $\mathcal{N}_4$, two machines i' and $i'', i' \neq i''$ are selected. Then, two random batches $\mathcal{B}' \in \Pi_{i'}$ and $\mathcal{B}'' \in \Pi_{i''}$ are directly swapped, that is, maintaining their initial positions.
- *Random job swap* ($\mathcal{N}_6$): Two random jobs from two different batches are selected. The movement is executed if the resulting batches are feasible. Otherwise, the old solution is maintained and the algorithm proceeds to another neighborhood.
- *Intensive job insertion* ($\mathcal{N}_7$): This neighborhood is inspired by the recent results reported by Medeiros et al. (2024). A randomly selected job j assigned to machine i' is removed from its batch. A machine i'' is then randomly chosen. Two moves are considered: (i) inserting j into all existing feasible batches of i'' that respect family and capacity constraints or (ii) creating a new singleton batch and inserting it before the first batch, after the last, or between existing batches. The best move is executed.
- *Intensive job swap* ($\mathcal{N}_8$): Two machines i' and i'' are randomly selected. Job j is also randomly selected from machine i'. Then, job j is swapped with all jobs from machine i'' that produce a feasible move. The best move is executed.
- *Intensive batch insertion* ($\mathcal{N}_9$): A batch $\mathcal{B}$ from machine i' is randomly selected and removed from its position in $\Pi_{i'}$. Then, $\mathcal{B}$ is inserted before the first batch, after the last, and between batches of a randomly chosen machine i''. The best move is executed.
- *Intensive batch swap* ($\mathcal{N}_{10}$): A batch $\mathcal{B}$ from machine i' is randomly selected and swapped with each batch from a randomly chosen machine i''. If $i' = i''$, then $\mathcal{B}$ is swapped with all other batches from the same machine sequence $\Pi_{i'}$. The best move is executed.

5.2. SLS framework

The SLS algorithm receives an initial solution generated by the constructive procedure described in Section 4 and iteratively tries to improve it by transitioning between neighborhoods. To obtain better results we introduce a certain degree of diversification that is controlled through a parameter $\theta \in [0, 1]$. This parameter establishes the probability of executing a *random perturbation* in the current solution. Moreover, we allow the acceptance of worsening moves using a linear SA criterion (Santini et al., 2018). In other words, the initial temperature T_0 gradually decays towards zero during the execution of the algorithm. This type of annealing scheme produced good results for similar problems (Franzin & Stützle, 2023; Rolim et al., 2023). T_0 is determined using the following relationship:

$$T_0 = \frac{0.1 \cdot Z}{\ln 2} \tag{15}$$

implying that the initial temperature is set such that there is a 50% chance of acceptance of a solution that is 10% worse than the initial solution objective Z obtained by the constructive method (Cota et al., 2017). Intuitively, the value of the current temperature T can be easily updated based on the current elapsed time τ and the imposed time limit $\tau^{\max}$:

$$T = T_0 \cdot \left(1 - \frac{\tau}{\tau^{\max}}\right) \tag{16}$$

Algorithm 2 shows the pseudocode of the SLS algorithm. We use Z, Z^{new}, and Z^{best} to designate the objective function values of the incumbent solution, of the new solution, and of the best solution, respectively.

Algorithm 2: Stochastic local search

Input : $\mathcal{F}$, $\mathcal{J}$, $\mathcal{M}$, $\mathcal{P}$, $\mathcal{S}$, $\mathcal{V}$, $v_{\max}$, θ
Output: A schedule Π^{best}, $\sum C_j$

$\Pi \leftarrow$ `Constructive_Heuristic()` ;
$\Pi^{\text{new}}, \Pi^{\text{best}} \leftarrow \Pi$;
Compute T_0;
while $\tau \leq \tau^{\max}$ **do**
  $r \leftarrow$ `rand()` ;
  `/* Random perturbation */`
  if $r < \theta$ **then**
    $\Pi^{\text{new}} \leftarrow$ `Perturbation()` ;
  end
  `/* Intensification */`
  $\Pi^{\text{new}} \leftarrow$ `Intensification()` ;
  $\Delta \leftarrow Z^{\text{new}} - Z$;
  `/* Acceptance criterion */`
  if $\Delta < 0$ **then**
    $\Pi \leftarrow \Pi^{\text{new}}$;
    if $Z^{new} < Z^{best}$ **then**
      $\Pi^{\text{best}} \leftarrow \Pi^{\text{new}}$;
    end
  else
    $r \leftarrow$ `rand()` ;
    if $r < \exp \frac{-\Delta}{T}$ **then**
      $\Pi \leftarrow \Pi^{\text{new}}$;
    else
      $\Pi^{\text{new}} \leftarrow \Pi$;
    end
  end
  `/* Temperature update */`
  $T \leftarrow T_0 \cdot \left(1 - \frac{\tau}{\tau^{\max}}\right)$;
end

6. Adaptive large neighborhood search

Originally developed for vehicle routing problems (Ropke & Pisinger, 2006), ALNS iteratively refines a solution produced by a constructive heuristic, employing pairs of removal and insertion neighborhoods based on the ruin-and-recreate principle. We follow a simplified framework inspired by the one proposed by Rolim et al. (2023) and show a set of operators designed to explore problem characteristics, including the relationships between batch formation, processing times, and machine capacity.

6.1. Removal and insertion neighborhoods

Let Ω^- and Ω^+ denote the sets of removal and insertion operators, respectively. At each iteration, a removal operator $\omega^- \in \Omega^-$ is chosen to extract elements from an incumbent solution, which are then stored in a list of removed jobs $\mathcal{R}$. Subsequently, an insertion operator $\omega^+ \in \Omega^+$ reconstructs the solution by reintegrating the elements from $\mathcal{R}$, producing a new feasible solution. The number of removed elements N^e is controlled by the parameter $q \in [0, 1]$, such that $N^e = \lceil q \cdot n \rceil$. As with the neighborhoods described in Section 5, we detail the removal and insertion heuristics as follows:

- *Random removal* (ω_1^-): Jobs are randomly selected and removed from the incumbent solution, following a structure similar to the removal operator in the IG algorithm proposed by Arroyo et al. (2019). If all jobs are removed from a batch, this batch is removed from the solution.
- *Maximum processing time removal* (ω_2^-): A batch $\mathcal{B}$ is randomly selected from the incumbent solution, and job j with the largest processing time, i.e., $\max_{j \in \mathcal{B}} p_{ij}$, is removed and added to $\mathcal{R}$. This approach targets jobs with higher processing times, as they determine the batch processing duration and can significantly impact the total sum of completion times.
- *Workload removal* (ω_3^-): The contribution of each machine to the objective value is computed. Next, a random job is removed from the machine with the highest workload, i.e., the machine with the largest sum of completion times. This procedure is repeated until all N^e jobs are removed.
- *Deviation removal* (ω_4^-): For each batch $\mathcal{B}$ with processing time P assigned to machine i, the sum of deviations for its jobs are calculated as $D = \sum_{j \in \mathcal{B}} (P - p_{ij})$. These deviations are computed for all batches in the current solution and stored in a list $\mathcal{L}$, which is then sorted in non-decreasing order of deviations. Random jobs are subsequently selected and removed from the top 20% of batches with the largest deviations. The goal is to prioritize the removal of jobs from dissimilar batches, as jobs with longer processing times can disproportionately penalize those with shorter processing times within the same batch.
- *Greedy insertion* (ω_1^+): Jobs are sequentially removed from $\mathcal{R}$ preserving the list's order. Next, they are inserted into the partial schedule using two strategies: (i) adding j to all feasible batches in the partial schedule or (ii) creating a new singleton batch and placing it in all possible positions within the schedule. The move that results in the best objective value is executed. This process is repeated until all jobs are scheduled.
- *SPT greedy insertion* (ω_2^+): This operator builds on ω_1^+, with the key difference that the list of removed jobs $\mathcal{R}$ is first sorted according to the SPT rule before performing the greedy insertion.
- *Competitive greedy insertion* (ω_3^+): Inspired by the operator proposed in Rolim et al. (2023), this method introduces a competition among the jobs in $\mathcal{R}$. Rather than inserting the jobs sequentially, all candidate jobs are evaluated using the greedy strategies outlined in ω_1^+. The job that results in the smallest cost increase is selected and scheduled. This process is repeated until $\mathcal{R} = \emptyset$.

Table 1
Scores of the weight update mechanism.

Parameter Ψ	Description
σ_1	The new solution is a global best.
σ_2	The new solution was accepted with a cost that is better than the cost of the current solution.
σ_3	The new solution was accepted with a cost that is worse than the cost of the current solution.
σ_4	The new solution is rejected.

- *Greedy batch insertion* (ω_4^+): This operator prioritizes inserting jobs into existing batches where the candidate job does not increase the batch processing time. Among the eligible batches, the job is inserted in the one that results in the smallest cost increase. If no such batch exists, a new singleton batch is created and inserted in the best possible position.

6.2. ALNS framework

Similar to the SLS, our ALNS adopts the linear SA criterion described in Section 5.2. Operator selection employs a probabilistic scheme guided by weights, using the roulette wheel mechanism. Following the approach proposed by Kovacs et al. (2012), we assign scores and weights to all possible pairs of removal and insertion operators, rather than treating them separately. In the beginning, the weights of all heuristic pairs are set to one. During execution time, they are updated at the end of every iteration with respect to a reaction factor $\rho \in [0, 1]$ and a score Ψ obtained by the corresponding pair. Let w_{ij} represent the weight of a removal/insertion pair i at iteration j. Therefore, at the end of each iteration, the weights are updated as follows:

$$w_{ij+1} = \rho \cdot w_{ij} + \Psi \cdot (1 - \rho) \tag{17}$$

The values of Ψ are determined using the SA acceptance criterion and are defined as shown in Table 1 (Sacramento et al., 2019). A detailed description of the ALNS is shown in Algorithm 3.

Algorithm 3: ALNS

Input : $\mathcal{F}$, $\mathcal{J}$, $\mathcal{M}$, $\mathcal{P}$, $\mathcal{S}$, $\mathcal{V}$, $v_{\max}$, q, ρ
Output: A schedule Π^{best}, $\sum C_j$

```
1  Π ← Constructive_Heuristic() ;
2  Π^new, Π^best ← Π;
3  Compute N^e and T_0;
4  while τ ≤ τ^max do
5      Select a removal ω⁻ ∈ Ω⁻ and an insertion ω⁺ ∈ Ω⁺ pair;
6      Remove N^e jobs from Π^new using ω⁻ and add them to R;
7      Reinsert the jobs in R into Π^new using ω⁺;
8      Δ ← Z^new − Z;
       /* Acceptance criterion                                  */
9      if Δ < 0 then
10         Π ← Π^new;
11         if Z^new < Z^best then
12             Π^best ← Π^new;
13         end
14     else
15         r ← rand() ;
16         if r < exp(−Δ/T) then
17             Π ← Π^new;
18         else
19             Π^new ← Π;
20         end
21     end
       /* Update the weights of the pair ω⁻ and ω⁺
          according to acceptance criterion                     */
22     Update_Weights();
       /* Temperature update                                    */
23     T ← T_0 · (1 − τ/τ^max);
24 end
```

7. Reinforcement learning for neighborhood selection

While exploring neighborhoods with equal probability is a common strategy in SLS algorithms, it can become a limitation in certain situations. Consequently, the effectiveness of specific neighborhoods can vary depending on problem characteristics. We implemented a simplified version of the Q-Learning algorithm to obtain online information on which neighborhood should be explored next. Here, we rely on the ideas discussed by dos Santos et al. (2014) to test whether adding Q-Learning algorithm to both SLS and ALNS has the potential of improving their performances.

In Q-Learning, an agent interacts with the environment and selects actions according to the rewards it collects. Each state–action pair $(t, a) \in S \times A$ is associated with a score $Q(t, a) \in \mathbb{R}$, representing the expected gain associated with the choice of a possible action a at state t:

$$Q(t, a) = Q(t, a) + \alpha \cdot [R(t, a) + \gamma \cdot \max_{a' \in A} Q(t', a') - Q(t, a)] \tag{18}$$

where t is the current state, a is the action taken in t, t' is the next state after performing action a, and a' is an action in state t'. Additionally, $\alpha \in [0, 1]$ is a parameter that defines the learning rate, $R(t, a)$ is the reward associated with action a in state t, and $\gamma \in [0, 1]$ is a discount factor that determines the importance of the future reward.

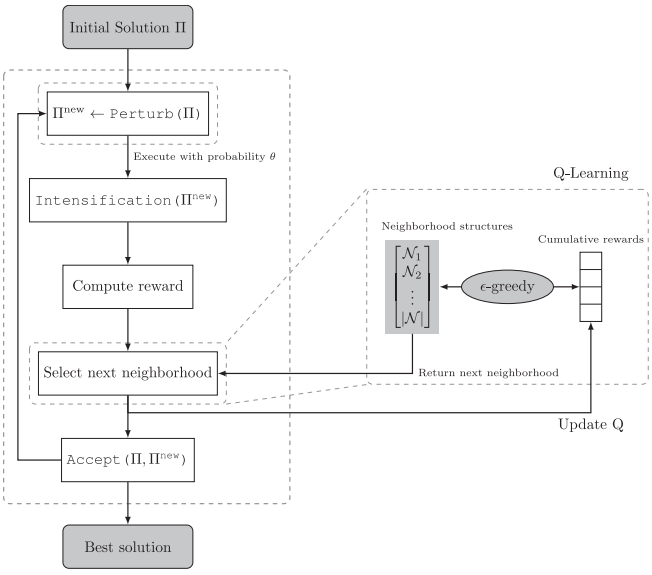
For the SLS, we adopt the approach by Alicastro et al. (2021) and Pugliese et al. (2022), where each neighborhood structure represents both actions and states. The set of actions is defined as $A = \{1, 2, \ldots, |\mathcal{N}|\}$, where $|\mathcal{N}|$ is the number of neighborhood structures in the intensification phase. In contrast, ALNS selects a pair of removal and insertion heuristics at each iteration. This distinction allows for a direct mapping between state–action pairs and removal–insertion heuristics. Accordingly, the set of actions can be defined as $A = \{1, 2, \ldots, |\Omega^+|\}$, and the set of states as $S = \{1, 2, \ldots, |\Omega^+|\}$ (or vice versa), where $|\Omega^-|$ and $|\Omega^+|$ represent the number of removal and insertion heuristics, respectively. An outline of the neighborhood selection procedure is given in Algorithm 4.

Algorithm 4: Q-Learning

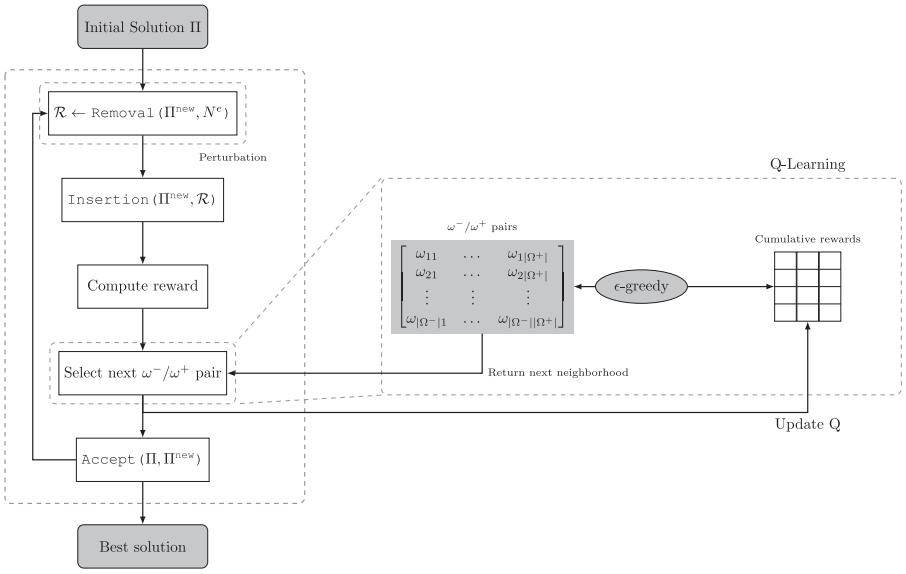
Input : Q, a, α, γ, ϵ, Z^{new}, Z
Output: Selected action

```
   /* Calculate the reward value of the current action */
1  R(t, a) ← max{0, Z − Z^new} ;
   /* Update Q-matrix                                  */
2  Q(t, a) ← Q(t, a) + α · [R(t, a) + γ · max_{a'∈A} Q(t', a') − Q(t, a)] ;
   /* Select the next action a' based on ε-greedy
      strategy                                         */
3  r ← rand() ;
4  if r < ε then
5      a' ← rand(N) ;
6  else
7      a' ← arg max_{a''∈A} Q(t', a'') ;
8  end
9  return a';
```

We note that an important aspect of the Q-Learning algorithm is making a balance between intensification and diversification (Karimi-Mamaghan et al., 2023). To avoid the algorithm from always choosing the neighborhood with the highest Q-value, we adopt an ϵ-greedy strategy. Therefore, we introduce a parameter ϵ that determines the probability of visiting other neighborhoods (line 4). The rearward function is configured to assume only positive values as the distance



(a) SLS_{RL} framework.



(b) $ALNS_{RL}$ framework.

Fig. 3. Flowcharts of neighborhood selection process using Q-Learning.

between the incumbent solution and the new solution in such a way that it is proportional to the amount of improvement (line 1).

Fig. 3 illustrates the neighborhood selection process for SLS_{RL} (Fig. 3(a)) and $ALNS_{RL}$ (Fig. 3(b)). From this point onward, we use this terminology to refer to the integration of Q-Learning with SLS and ALNS, respectively. At each iteration, the algorithms use historical information obtained from exploring a specific intensification neighborhood or a removal/insertion heuristic pair to guide future selections.

8. Computational experiments

For generating the problem instances, we followed most of the recommendations described by Gahm et al. (2021). The parameters of each instance are the number of machines m, the number of jobs n, the number of incompatible job families f, the maximum machine capacity v_{max}, the set of job sizes $\mathcal{V}$, the set of processing times $\mathcal{P}$, and the set of setup times $\mathcal{S}$. We divided the instance set into three categories denoted as C_1, C_2, C_3. The first category is used for validation purposes, the second contains small-to-medium instances with up to 60 jobs, and the third contains large instances with up to 400 jobs. For each combination, we generated a total of $K = 10$ different instances.

The machine capacity is assumed to be the same $v_{max} = 50$, however, we consider two different capacity-requirement scenarios $v_j \sim U[1, 25]$ and $v_j \sim U[13, 38]$. Processing times are also drawn from a discrete uniform distribution from the interval $p_{ij} \sim U[1, 100]$. For generating the setup-times, which are both machine and sequence dependent, we first compute the mean batch size of a problem instance as:

$$\bar{v} = \frac{\sum_{j \in \mathcal{J}} v_j}{n} \tag{19}$$

with this information, the mean number of jobs per batch is approximated using the following relationship:

$$\bar{n} = \left\lceil \frac{v_{max}}{\bar{v}} \right\rceil \tag{20}$$

Table 2
Configuration of benchmark instances.

Parameter	Instance set		
	C_1	C_2	C_3
m	{2}	{2, 4, 6}	{8, 10, 12}
n	{10}	{20, 40, 60}	{100, 200, 400}
f	{2, 4, 6}	{2, 4, 6}	{2, 4, 6}
v_j	$\sim U[1, 25]$	$\sim U[1, 25]$	$\sim U[1, 25]; \sim U[13, 38]$
$v_{\max}$	{50}	{50}	{50}
η	{0.25}	{0.25}	{0.25, 0.75}
K	{10}	{10}	{10}
Total:	30	270	1080

then, the mean processing time per batch is determined as $\bar{P} = \bar{n} \cdot \bar{p}$ with the understanding that:

$$\bar{p} = \left\lceil \frac{\sum_{j \in \mathcal{J}} \max_{i \in \mathcal{M}} p_{ij}}{n} \right\rceil \tag{21}$$

Finally, a setup severity factor η is considered to generate the mean setup time $\bar{s} = \eta \cdot \bar{P}$ such that $s_{ikl} \sim U[1, 2\bar{s}]$. We note that, in contrast with Gahm et al. (2021), we do not impose the setup times to respect the triangle inequality. Table 2 shows the characteristics of each instance category. All instances and detailed results are publicly available as supplementary material in Rolim et al. (2024b). A reproducible capsule with source codes can be executed in Rolim et al. (2024a).

All algorithms were implemented in Julia 1.7.3, and computational experiments were conducted on a cluster featuring nodes with two Intel® Xeon™ CPU E5-2680v2 at 2.8 GHz, 128 GB 1866 MHz RAM, and CentOS Linux 7.2.1511 as the operating system. The MIP was formulated using JuMP (Dunning et al., 2017) and solved using Gurobi™ 10.0.0. We report MIP results only for category C_1 due to a rapid decline in solution quality with increasing problem size even within a 3600-s time limit. Furthermore, we conducted five independent runs for each metaheuristic and the maximum allowed CPU time is determined as $\tau^{\max} = m \cdot n \cdot t/2$ milliseconds, where $t = 90$ (Vallada & Ruiz, 2011). Instead of retaining the best objective values, we calculated their average to derive the relative percentage deviation (RPD), expressed as:

$$\text{RPD} = \left(\frac{\bar{Z}^{\text{heu}} - Z^{\text{best}}}{Z^{\text{best}}} \right) \cdot 100\% \tag{22}$$

Here, Z^{best} and $\bar{Z}^{\text{heu}}$ represent the best objective value and the average objective value of a given metaheuristic, respectively. We also reinforce that we followed most of the optimization best practices reported by Kendall et al. (2016).

8.1. Parameter configuration

The investigated problem lacks established heuristics in the literature. However, some existing methods are readily adaptable. To benchmark the performance of both ALNS and SLS, we implemented the VNS from Cakici et al. (2013) and the IG introduced by Arroyo et al. (2019). Specifically, we focused on replicating the most recent version of the IG proposed by Jiang et al. (2022). The selection of these methods considered the machine environment (parallel machines), job family restrictions, and the objective function (minimizing total completion time).

Designing efficient metaheuristics involves the challenge of identifying their most effective components. Although incorporating additional elements to address their specific weaknesses can occasionally enhance performance, it may also introduce drawbacks such as a larger parameter space, increased computational cost, and potential scalability issues. To address these problems, we decompose SLS and ALNS into multiple variants.

We propose three versions of SLS. The first, integrates the SA acceptance criterion as described in Section 5. The second, SLS_{L}, incorporates the learning mechanism of ALNS into SLS following the weight structure in Table 1. Lastly, the third, SLS_{RL}, integrates the scheme detailed in Fig. 3(a). For ALNS, we examine two variants: the first follows the standard framework outlined in Section 6, utilizing its conventional learning mechanism, while the second, ALNS_{RL} incorporates mechanism described in Fig. 3(b).

Table 3
Parameter configuration obtained by `irace`.

Algorithm	Parameter	Type	Interval	Suggested values
VNS	$k_{\max}$	Categorical	{1, 2, 3, 4, 5}	5
	n_{iter}	Categorical	{25, 50, 100}	100
IG	nD	Categorical	{0.05, 0.08, 0.1, 0.2}	0.05
	$Prob$	Categorical	{0.0, 0.05, 0.1}	0.05
	$dist$	Categorical	{1, 2, 3, 4, 5, 6}	5
	LC	Categorical	{1, 10, 50, 100, 150, 200}	1
SLS	θ	Real	[0, 1]	0.13
SLS_{L}	θ	Real	[0, 1]	0.01
	ρ	Real	[0, 1]	0.98
	σ_1	Integer	[0, 35]	19
	σ_2	Integer	[0, 35]	35
	σ_3	Integer	[0, 35]	18
SLS_{RL}	θ	Real	[0, 1]	0.11
	α	Real	[0, 1]	0.61
	ϵ	Real	[0, 1]	0.45
	γ	Real	[0, 1]	0.64
ALNS	q	Categorical	{0.05, 0.08, 0.1, 0.2}	0.05
	ρ	Real	[0, 1]	0.02
	σ_1	Integer	[0, 35]	22
	σ_2	Integer	[0, 35]	2
	σ_3	Integer	[0, 35]	6
ALNS_{RL}	q	Categorical	{0.05, 0.08, 0.1, 0.2}	0.08
	α	Real	[0, 1]	0.22
	ϵ	Real	[0, 1]	0.02
	γ	Real	[0, 1]	0.17

The parameter configuration for each algorithm was determined using the `irace` package (López-Ibáñez et al., 2016). To mitigate the risk of over-fitting, we employed a hold-out validation strategy. This involved randomly generating a set of training instances with diverse characteristics. These training instances are readily available alongside the other three instance categories for further reference. We set each target algorithm to run 1000 times with the same stopping criterion defined for the test instances. Table 3 shows the design of experiments as well as the final configurations suggested by `irace`.

8.2. Results and discussion

We compare our approaches to established methods on categories C_2 and C_3. Building upon the algorithm proposed by Arroyo et al. (2019), the IG demonstrably outperforms many other metaheuristics, as empirical evidence suggests. Therefore, IG is considered the current state-of-the-art for solving parallel batch scheduling problems. Since VNS has not been previously compared to IG, we include it as an additional benchmark. We also observe that, setting the maximum elapsed time as the stopping criterion permits a fair comparison since the number of iterations between metaheuristics may vary substantially under the same time budget (Ravber et al., 2022).

Tables 4 and 5 present the average RPD achieved by each metaheuristic on instances of categories C_2 and C_3, respectively. These values are segmented based on instance size ($m \times n$) and the number of job families (f). Best results within each size-family combination are highlighted in boldface. Additionally, minimum, average, and maximum RPD values for the entire set, along with the standard deviation ($\bar{\sigma}$), are reported. We first discuss the results of the proposed methods against the benchmark heuristics. Subsequently, we analyze the impact of the adaptive mechanisms.

Table 4
Average RPD (%) values on instances from C_2.

$m \times n$	f	VNS	IG	SLS	SLS_{RL}	SLS_{L}	ALNS	$ALNS_{RL}$
2 × 20	2	17.968	11.764	0.227	0.110	**0.073**	2.061	0.173
	4	11.216	14.632	0.023	0.044	0.071	1.475	**0.005**
	6	10.707	10.031	0.153	0.020	0.114	0.685	**0.017**
4 × 20	2	42.891	5.224	0.267	0.368	0.154	0.458	**0.000**
	4	38.272	8.519	0.122	0.109	0.232	0.493	**0.000**
	6	36.836	5.247	0.245	0.255	0.260	0.431	**0.000**
6 × 20	2	68.757	1.684	0.124	0.135	0.148	0.121	**0.000**
	4	58.369	2.983	0.071	0.102	0.132	0.055	**0.000**
	6	52.001	2.685	0.292	0.232	0.412	0.142	**0.000**
2 × 40	2	15.321	8.586	1.702	**1.346**	1.730	4.682	2.346
	4	19.480	9.082	1.508	1.489	**1.158**	4.483	2.040
	6	21.024	6.147	1.367	1.491	**0.857**	3.162	1.670
4 × 40	2	30.709	1.941	1.389	1.449	1.311	0.827	**0.521**
	4	36.980	2.545	1.962	1.900	1.527	0.956	**0.209**
	6	39.860	1.367	1.614	1.560	1.124	0.819	**0.094**
6 × 40	2	55.568	0.201	1.497	1.282	1.224	0.397	**0.028**
	4	59.008	0.865	1.414	1.288	1.545	0.512	**0.073**
	6	52.803	1.618	1.507	1.631	1.728	0.189	**0.015**
2 × 60	2	18.034	8.027	2.577	2.630	**2.378**	6.187	6.518
	4	20.130	7.876	2.073	1.930	**1.588**	5.800	5.349
	6	20.847	8.550	1.806	1.980	**1.652**	6.475	5.338
4 × 60	2	32.911	2.701	2.090	2.078	**1.951**	2.997	2.676
	4	35.349	2.598	2.988	3.122	2.558	2.192	**1.629**
	6	37.833	2.765	2.974	3.773	3.107	1.539	**1.363**
6 × 60	2	50.693	**0.935**	3.142	2.628	2.592	1.074	1.020
	4	58.109	0.617	2.948	3.057	3.034	1.361	**0.558**
	6	51.241	1.530	2.945	3.286	3.129	**0.572**	0.654
Min.		5.011	0.000	0.000	0.000	0.000	0.000	0.000
Average		36.829	4.802	1.445	1.457	1.327	1.856	1.196
Max.		103.350	32.600	5.804	7.743	5.744	13.914	10.706
$\bar{\sigma}$		20.157	5.868	1.333	1.371	1.232	2.534	2.016

Proposed heuristics. Overall, both ALNS and SLS variants outperformed the benchmark heuristics, albeit at the cost of incorporating a larger number of neighborhood structures. Table 4 highlights that $ALNS_{RL}$ emerged as the best-performing algorithm for instance set C_2, achieving an average RPD of 1.196%. The SLS variants followed closely, with average RPDs below 1.500%, while the standard ALNS attained an average RPD of 1.856%. Notably, the inclusion of the adaptive layer in SLS does not seem to significantly enhance solution quality for category C_2. Specifically, SLS_{L} achieves only marginal gains, with an improvement of 0.118% over the conservative SLS and 0.130% over SLS_{RL}. Table 5 reveals that as instance size increases, the standard ALNS emerges as the best-performing heuristic, achieving an average RPD of 0.969%. It is followed by $ALNS_{RL}$ with an average RPD of 1.406%, SLS_{L} with 8.106%, $SLS_{RL}$with 8.480%, and SLS with 8.762%.

Benchmark heuristics. Table 4 reveals that VNS exhibited the highest average RPD (36.829%) and standard deviation ($\bar{\sigma} = 20.157\%$) among the compared methods. A closer examination shows that VNS performs relatively better in scenarios with fewer machines, as increasing the values of m while keeping n and f constant results in higher average RPD values. In contrast, IG demonstrated strong performance on instances from category C_2, achieving an average RPD of 4.802%. This performance significantly outperformed VNS, with best results observed in the 6×60 instances, particularly when $f = 2$ and $f = 4$. For instance set C_3 (see Table 5), both VNS and IG experienced a noticeable decline in solution quality, as reflected in their increased average RPD values.

Impact of Q-learning. A deeper analysis of the computational experiments suggests that, although $ALNS_{RL}$ achieved the best results on instance set C_2, the benefits of Q-Learning over the classic adaptive layer of ALNS remain unclear. This is due to its performance degradation on larger-scale instances in C_3, suggesting limitations in scalability. Moreover, the adaptive layers in both SLS_{RL} and SLS_{L} produced only minimal performance gains. We conjecture that the importance of the adaptive layer depends on several factors such as the types of neighborhoods explored, their interaction with the objective function, the reward or weight adjustment mechanism, and parameter calibration. It is worth noting that Q-Learning should be employed with care, as it increases the parameter space and makes the tuning process more complicated, even when relying on automatic algorithm configuration. Silberholz et al. (2019) offer an interesting discussion on this issue, emphasizing that the parameter space grows exponentially with the number of parameters. Consequently, simpler algorithms have a higher chance of scaling better, specially on instances with different characteristics. These findings align with the notion of the *less-is-more* approach, emphasizing that adding new components to metaheuristics incurs additional costs and does not always lead to significant improvements over simpler methods.

8.3. Time-to-target analysis

To gain deeper insights into the algorithms' efficiency, we additionally conducted a time-to-target experiment on multiple problem instances (Reyes & Ribeiro, 2018). This experiment assesses the likelihood of an algorithm discovering a solution that meets or surpasses a predetermined target value within a specific time limit for an instance or a group of instances. In simpler terms, it evaluates the speed at which the algorithm converges towards a solution satisfying a certain quality threshold. For this analysis, we have considered the ten instances from C_3 with the following characteristics: $m = 12$, $n = 200$, $f = 6$, $v_j \sim U[13, 38]$, and $\eta = 0.75$. The target solutions were set as the median from the five executions of IG. Each algorithm was executed 120 times for each instance-target pair, until a solution as good as the corresponding target was found or 500 s of elapsed CPU time was reached. The multiple time-to-target (MTTT) plots are depicted in Fig. 4.

The MTTT analysis observed in Fig. 4 support the trends discussed in Section 8.2. Specifically, IG can take almost 3000 s to reach a solution as good as the target across the 10 selected instances. In contrast, both

Table 5
Average RPD (%) values on instances from C_3.

$m \times n$	f	VNS	IG	SLS	SLS_{RL}	SLS_{L}	ALNS	$ALNS_{RL}$
	2	36.224	2.025	4.018	3.901	3.146	**0.941**	1.307
8 × 100	4	39.996	2.222	4.970	4.614	4.164	**0.765**	1.016
	6	44.204	2.503	5.162	4.757	5.092	**0.877**	0.939
	2	47.848	2.375	4.546	4.523	3.813	0.665	**0.945**
10 × 100	4	52.209	2.212	5.273	4.850	5.148	**0.625**	0.703
	6	55.181	2.277	5.603	5.122	5.964	0.746	**0.683**
	2	59.246	1.990	5.217	4.906	4.506	**0.540**	0.629
12 × 100	4	59.794	2.107	5.841	5.444	5.833	**0.338**	0.491
	6	61.001	2.290	6.009	5.435	6.997	**0.427**	0.546
	2	25.560	21.081	6.590	6.485	5.347	**1.015**	1.988
8 × 200	4	33.348	21.447	8.870	8.427	7.394	**1.082**	1.942
	6	37.616	22.565	10.682	10.240	8.981	**1.171**	1.796
	2	35.441	23.383	8.192	8.154	6.502	**1.066**	1.979
10 × 200	4	43.161	24.777	10.752	10.189	8.870	**0.998**	1.513
	6	47.459	25.177	12.221	11.437	11.225	**0.913**	1.494
	2	45.291	25.196	9.842	9.568	8.214	**0.605**	1.513
12 × 200	4	51.720	26.879	11.902	11.384	10.770	**0.751**	1.620
	6	57.969	26.962	13.579	12.676	12.487	**0.870**	1.200
	2	27.695	31.453	8.872	8.608	7.671	**1.278**	1.757
8 × 400	4	26.552	44.139	9.451	9.200	8.518	**1.401**	1.696
	6	25.556	57.726	9.616	9.538	9.045	**1.503**	1.674
	2	25.717	48.865	9.162	9.117	8.756	**1.120**	1.605
10 × 400	4	34.148	70.222	11.919	11.803	11.412	**1.224**	1.654
	6	40.685	72.612	12.704	12.686	12.490	**1.355**	1.957
	2	33.422	47.028	9.940	9.918	9.765	**0.960**	1.742
12 × 400	4	41.152	63.275	10.919	10.928	10.874	**1.085**	1.824
	6	48.479	71.355	14.411	14.414	14.307	**1.204**	1.800
Min.		5.224	0.241	0.832	0.832	0.831	0.013	0.083
Average		42.039	29.650	8.762	8.480	8.106	0.969	1.406
Max.		140.541	114.332	25.624	24.746	24.150	6.104	6.359
$\bar{\sigma}$		25.079	27.367	4.304	4.222	4.053	0.793	0.801

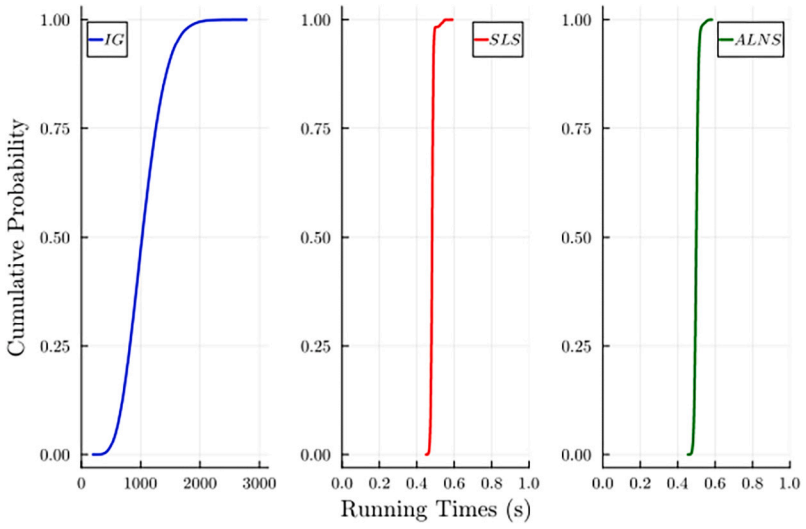


Fig. 4. MTTT plots of IG, SLS and ALNS with $1.2 \cdot 10^5$ points generated by simulation.

SLS and ALNS demonstrate significantly faster convergence, requiring between 0.4 and 0.6 s to reach the same targets. This substantial difference can be attributed, at least partially, to the larger set of neighborhood structures employed by SLS and ALNS. Notably, despite their comparable convergence profiles relative to IG, ALNS performed better than SLS on C_3, demonstrating a greater ability to escape local optima within the same time budget.

9. Conclusions

This paper tackles an unrelated parallel batch scheduling problem with family setups, a problem with many practical applications. The processing time for each job is determined by a p-batch assumption, and the objective is to minimize the total sum of job completion times.

Since this problem is strongly $\mathcal{NP}$-hard for the more specific single-machine case (Yang et al., 2022), we proposed a polynomial-time heuristic as well as two metaheuristics, namely, SLS and ALNS. Furthermore, we explored various neighborhood structures and examined whether incorporating adaptive layers based on Q-Learning and on the standard ALNS scheme improves performance.

Our results suggest that, while integrating Q-Learning for neighborhood selection can sometimes produce better results, it also increases the complexity of the algorithms. Consequently, finding a reasonable parameter configuration can be complex, even when using `irace`. Under the same experimental conditions, the conservative approaches of SLS and ALNS presented consistent results across the test instances in comparison with well-established algorithms from literature.

Several promising research directions emerge from this work. One possible direction is to evaluate the use of deep reinforcement learning as a hyper-heuristic framework for ALNS to solve a broader class of related problems that involves both batching and scheduling decisions. Recently, Kallestad et al. (2023) presented an interesting study in this direction. Another possible line includes the use of constraint programming (CP) and mathematical programming, as well as their interplay with metaheuristics (Mönch & Roob, 2018). In terms of new problems, it would be interesting to explore the coordinated integration of BPMs in supply chains (Jula & Leachman, 2010) or address challenges in additive manufacturing systems (Alicastro et al., 2021). Finally, a valuable contribution to the field would be a meta-analysis on the impact of Q-Learning within metaheuristics, a line preliminarily explored by Turkeš et al. (2021) in the context of the adaptive layer of ALNS. Such an approach could yield stronger empirical evidence on the trade-offs associated with Q-Learning integration.

CRediT authorship contribution statement

Gustavo Alencar Rolim: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Caio Paziani Tomazella:** Writing – review & editing, Validation, Methodology, Conceptualization. **Marcelo Seido Nagano:** Writing – review & editing, Project administration, Conceptualization.

Acknowledgments

This work was supported by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), Brazil - Finance Code 001 [grant number 88887.666292/2022-0], Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), Brazil [grant numbers 312585/2021-7 and 404819/2023-0], and by the São Paulo Research Foundation (FAPESP), Brasil [process number 2019/10824-7]. Research carried out using the computational resources of the Center for Mathematical Sciences Applied to Industry (CeMEAI) funded by FAPESP, Brazil [grant number 2013/07375-0]. We also thank Dr. Anselmo Ramalho Pitombeira Neto and the anonymous referees for the comments that helped improve the quality of the paper.

References

Alicastro, M., Ferone, D., Festa, P., Fugaro, S., & Pastore, T. (2021). A reinforcement learning iterated local search for makespan minimization in additive manufacturing machine scheduling problems. *Computers & Operations Research*, *131*, Article 105272. http://dx.doi.org/10.1016/j.cor.2021.105272.

Allahverdi, A., Gupta, J. N. D., & Aldowaisan, T. (1999). A review of scheduling research involving setup considerations. *Omega*, *27*(2), 219–239. http://dx.doi.org/10.1016/s0305-0483(98)00042-5.

Arroyo, J. E. C., & Leung, J. Y. T. (2017). Scheduling unrelated parallel batch processing machines with non-identical job sizes and unequal ready times. *Computers & Operations Research*, *78*, 117–128. http://dx.doi.org/10.1016/j.cor.2016.08.015.

Arroyo, J. E. C., Leung, J. Y. T., & Tavares, R. G. (2019). An iterated greedy algorithm for total flow time minimization in unrelated parallel batch machines with unequal job release times. *Engineering Applications of Artificial Intelligence*, *77*, 239–254. http://dx.doi.org/10.1016/j.engappai.2018.10.012.

Brimberg, J., Salhi, S., Todosijević, R., & Urošević, D. (2023). Variable neighborhood search: The power of change and simplicity. *Computers & Operations Research*, *155*, Article 106221. http://dx.doi.org/10.1016/j.cor.2023.106221.

Cakici, E., Mason, S. J., Fowler, J. W., & Geismar, H. N. (2013). Batch scheduling on parallel machines with dynamic job arrivals and incompatible job families. *International Journal of Production Research*, *51*(8), 2462–2477. http://dx.doi.org/10.1080/00207543.2012.748227.

Chandru, V., Lee, C. Y., & Uzsoy, R. (1993). Minimizing total completion time on batch processing machines. *International Journal of Production Research*, *31*(9), 2097–2121. http://dx.doi.org/10.1080/00207549308956847.

Chen, P., & Wang, Q. (2024). Learning for multiple purposes: A Q-learning enhanced hybrid metaheuristic for parallel drone scheduling traveling salesman problem. *Computers & Industrial Engineering*, *187*, Article 109851. http://dx.doi.org/10.1016/j.cie.2023.109851.

Cheng, B., Yang, S., Hu, X., & Chen, B. (2012). Minimizing makespan and total completion time for parallel batch processing machines with non-identical job sizes. *Applied Mathematical Modelling*, *36*(7), 3161–3167. http://dx.doi.org/10.1016/j.apm.2011.09.061.

Cota, L. P., Guimaraes, F. G., de Oliveira, F. B., & Souza, M. J. F. (2017). An adaptive large neighborhood search with learning automata for the unrelated parallel machine scheduling problem. In *2017 IEEE congress on evolutionary computation*. IEEE, http://dx.doi.org/10.1109/cec.2017.7969312.

Crauwels, H. A. J., Hariri, A. M. A., Potts, C. N., & Van Wassenhove, L. N. (1998). Branch and bound algorithms for single-machine scheduling with batch set-up times to minimize total weighted completion times. *Annals of Operations Research*, *83*, 59–76. http://dx.doi.org/10.1023/a:1018920416308.

Dobson, G., & Nambimadom, R. S. (2001). The batch loading and scheduling problem. *Operations Research*, *49*(1), 52–65. http://dx.doi.org/10.1287/opre.49.1.52.11189.

dos Santos, J. a. P. Q., de Melo, J. D., Duarte-Neto, A. a. D., & Aloise, D. (2014). Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. *Expert Systems with Applications*, *41*(10), 4939–4949. http://dx.doi.org/10.1016/j.eswa.2014.01.040.

Dunning, I., Huchette, J., & Lubin, M. (2017). JuMP: A modeling language for mathematical optimization. *SIAM Review*, *59*(2), 295–320. http://dx.doi.org/10.1137/15m1020575.

Dunstall, S., Wirth, A., & Baker, K. (2000). Lower bounds and algorithms for flowtime minimization on a single machine with set-up times. *Journal of Scheduling*, *3*(1), 51–69. http://dx.doi.org/10.1002/(sici)1099-1425(200001/02)3:1<51::aid-jos34>3.0.co;2-k.

Fowler, J. W., & Mönch, L. (2022). A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research*, *298*(1), 1–24. http://dx.doi.org/10.1016/j.ejor.2021.06.012.

Franzin, A., & Stützle, T. (2023). A landscape-based analysis of fixed temperature and simulated annealing. *European Journal of Operational Research*, *304*(2), 395–410. http://dx.doi.org/10.1016/j.ejor.2022.04.014.

Gahm, C., Wahl, S., & Tuma, A. (2021). Scheduling parallel serial-batch processing machines with incompatible job families, sequence-dependent setup times and arbitrary sizes. *International Journal of Production Research*, 1–24. http://dx.doi.org/10.1080/00207543.2021.1951446.

Gokhale, R., & Mathirajan, M. (2011). Heuristic algorithms for scheduling of a batch processor in automobile gear manufacturing. *International Journal of Production Research*, *49*(10), 2705–2728. http://dx.doi.org/10.1080/00207541003720368.

Graham, R., Lawler, E., Lenstra, J., & Kan, A. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. *vol. 5*, In *Annals of discrete mathematics* (pp. 287–326). Elsevier, http://dx.doi.org/10.1016/s0167-5060(08)70356-x.

Ham, A., Fowler, J. W., & Cakici, E. (2017). Constraint programming approach for scheduling jobs with release times, non-identical sizes, and incompatible families on parallel batching machines. *IEEE Transactions on Semiconductor Manufacturing*, *30*(4), 500–507. http://dx.doi.org/10.1109/tsm.2017.2740340.

Hochbaum, D. S., & Landy, D. (1997). Scheduling semiconductor burn-in operations to minimize total flowtime. *Operations Research*, *45*(6), 874–885. http://dx.doi.org/10.1287/opre.45.6.874.

Hoos, H. H., & Stützle, T. (2015). Stochastic local search algorithms: An overview. In *Springer handbook of computational intelligence* (pp. 1085–1105). Springer Berlin Heidelberg, http://dx.doi.org/10.1007/978-3-662-43505-2_54.

Ikura, Y., & Gimple, M. (1986). Efficient scheduling algorithms for a single batch processing machine. *Operations Research Letters*, *5*(2), 61–65. http://dx.doi.org/10.1016/0167-6377(86)90104-5.

Jia, Z. H., Zhang, H., tao Long, W., Leung, J. Y. T., Li, K., & Li, W. (2018). A meta-heuristic for minimizing total weighted flow time on parallel batch machines. *Computers & Industrial Engineering*, *125*, 298–308. http://dx.doi.org/10.1016/j.cie.2018.08.009.

Jiang, W., Shen, Y., Liu, L., Zhao, X., & Shi, L. (2022). A new method for a class of parallel batch machine scheduling problem. *Flexible Services and Manufacturing Journal*, *34*(2), 518–550. http://dx.doi.org/10.1007/s10696-021-09415-w.

Jula, P., & Leachman, R. C. (2010). Coordinated multistage scheduling of parallel batch-processing machines under multiresource constraints. *Operations Research*, *58*(4-part-1), 933–947. http://dx.doi.org/10.1287/opre.1090.0788.

Kallestad, J., Hasibi, R., Hemmati, A., & Sörensen, K. (2023). A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. *European Journal of Operational Research*, *309*(1), 446–468. http://dx.doi.org/10.1016/j.ejor.2023.01.017.

Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A. M., & Talbi, E.-G. (2022). Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research*, *296*(2), 393–422. http://dx.doi.org/10.1016/j.ejor.2021.04.032.

Karimi-Mamaghan, M., Mohammadi, M., Pasdeloup, B., & Meyer, P. (2023). Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, *304*(3), 1296–1330. http://dx.doi.org/10.1016/j.ejor.2022.03.054.

Kayhan, B. M., & Yildiz, G. (2023). Reinforcement learning applications to machine scheduling problems: a comprehensive literature review. *Journal of Intelligent Manufacturing*, *34*(3), 905–929. http://dx.doi.org/10.1007/s10845-021-01847-3.

Kendall, G., Bai, R., Błazewicz, J., Causmaecker, P. D., Gendreau, M., John, R., Li, J., McCollum, B., Pesch, E., Qu, R., Sabar, N., Vanden Berghe, G., & Yee, A. (2016). Good laboratory practice for optimization research. *Journal of the Operational Research Society*, *67*(4), 676–689. http://dx.doi.org/10.1057/jors.2015.77.

Kim, Y. D., Joo, B. J., & Choi, S. Y. (2010). Scheduling wafer lots on diffusion machines in a semiconductor wafer fabrication facility. *IEEE Transactions on Semiconductor Manufacturing*, *23*(2), 246–254. http://dx.doi.org/10.1109/tsm.2010.2045666.

Klemmt, A., Weigert, G., & Werner, S. (2011). Optimisation approaches for batch scheduling in semiconductor manufacturing. *European Journal of Industrial Engineering*, *5*(3), 338. http://dx.doi.org/10.1504/ejie.2011.041621.

Koh, S. G., Koo, P. H., Ha, J. W., & Lee, W. S. (2004). Scheduling parallel batch processing machines with arbitrary job sizes and incompatible job families. *International Journal of Production Research*, *42*(19), 4091–4107. http://dx.doi.org/10.1080/00207540410001704041.

Kovacs, A. A., Parragh, S. N., Doerner, K. F., & Hartl, R. F. (2012). Adaptive large neighborhood search for service technician routing and scheduling problems. *Journal of Scheduling*, *15*(5), 579–600. http://dx.doi.org/10.1007/s10951-011-0246-9.

Kramer, A., Iori, M., & Lacomme, P. (2021). Mathematical formulations for scheduling jobs on identical parallel machines with family setup times and total weighted completion time minimization. *European Journal of Operational Research*, *289*(3), 825–840. http://dx.doi.org/10.1016/j.ejor.2019.07.006.

Li, X. L., Huang, Y. L., Tan, Q., & Chen, H. P. (2013). Scheduling unrelated parallel batch processing machines with non-identical job sizes. *Computers & Operations Research*, *40*(12), 2983–2990. http://dx.doi.org/10.1016/j.cor.2013.06.016.

Li, R., Tan, Z., & Zhu, Q. (2021). Batch scheduling of nonidentical job sizes with minsum criteria. *Journal of Combinatorial Optimization*, *42*(3), 543–564. http://dx.doi.org/10.1007/s10878-019-00419-9.

López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., & Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, *3*, 43–58. http://dx.doi.org/10.1016/j.orp.2016.09.002.

Mason, A. J., & Anderson, E. J. (1991). Minimizing flow time on a single machine with job classes and setup times. *Naval Research Logistics*, *38*(3), 333–350. http://dx.doi.org/10.1002/1520-6750(199106)38:3<333::aid-nav3220380305>3.0.co;2-0.

Mathirajan, M., & Sivakumar, A. I. (2006). A literature review, classification and simple meta-analysis on scheduling of batch processors in semiconductor. *International Journal of Advanced Manufacturing Technology*, *29*(9–10), 990–1001. http://dx.doi.org/10.1007/s00170-005-2585-1.

Medeiros, J. M. F., Subramanian, A., & Queiroga, E. (2024). Population-based iterated local search for batch scheduling on parallel machines with incompatible job families, release dates, and tardiness penalties. *Optimization Letters*, http://dx.doi.org/10.1007/s11590-024-02116-x.

Mladenović, N., Pei, J., Pardalos, P. M., & Urošević, D. (2022). Less is more approach in optimization: a road to artificial intelligence. *Optimization Letters*, *16*(1), 409–420. http://dx.doi.org/10.1007/s11590-021-01818-w.

Mönch, L., & Roob, S. (2018). A matheuristic framework for batch machine scheduling problems with incompatible job families and regular sum objective. *Applied Soft Computing*, *68*, 835–846. http://dx.doi.org/10.1016/j.asoc.2017.10.028.

Monma, C. L., & Potts, C. N. (1989). On the complexity of scheduling with batch setup times. *Operations Research*, *37*(5), 798–804. http://dx.doi.org/10.1287/opre.37.5.798.

Ozturk, O. (2020). A truncated column generation algorithm for the parallel batch scheduling problem to minimize total flow time. *European Journal of Operational Research*, *286*(2), 432–443. http://dx.doi.org/10.1016/j.ejor.2020.03.044.

Ozturk, O., Espinouse, M. L., Di Mascolo, M., & Gouin, A. (2012). Makespan minimisation on parallel batch processing machines with non-identical job sizes and release dates. *International Journal of Production Research*, *50*(20), 6022–6035. http://dx.doi.org/10.1080/00207543.2011.641358.

Pinedo, M. L. (2022). *Scheduling: Theory, algorithms, and systems*. Springer International Publishing, http://dx.doi.org/10.1007/978-3-031-05921-6.

Potts, C. N., & Kovalyov, M. Y. (2000). Scheduling with batching: A review. *European Journal of Operational Research*, *120*(2), 228–249. http://dx.doi.org/10.1016/s0377-2217(99)00153-8.

Pugliese, L. D., Ferone, D., Festa, P., Guerriero, F., & Macrina, G. (2022). Combining variable neighborhood search and machine learning to solve the vehicle routing problem with crowd-shipping. *Optimization Letters*, *17*(9), 1981–2003. http://dx.doi.org/10.1007/s11590-021-01833-x.

Queiroga, E., Pinheiro, R. G. S., Christ, Q., Subramanian, A., & Pessoa, A. A. (2021). Iterated local search for single machine total weighted tardiness batch scheduling. *Journal of Heuristics*, *27*(3), 353–438. http://dx.doi.org/10.1007/s10732-020-09461-x.

Ravber, M., Liu, S.-H., Mernik, M., & Črepinšek, M. (2022). Maximum number of generations as a stopping criterion considered harmful. *Applied Soft Computing*, *128*, Article 109478. http://dx.doi.org/10.1016/j.asoc.2022.109478.

Reyes, A., & Ribeiro, C. C. (2018). Extending time-to-target plots to multiple instances. *International Transactions in Operational Research*, *25*(5), 1515–1536. http://dx.doi.org/10.1111/itor.12507.

Rolim, G. A., & Nagano, M. S. (2020). Structural properties and algorithms for earliness and tardiness scheduling against common due dates and windows: A review. *Computers & Industrial Engineering*, *149*, Article 106803. http://dx.doi.org/10.1016/j.cie.2020.106803.

Rolim, G. A., Nagano, M. S., & Prata, B. d. A. (2023). Formulations and an adaptive large neighborhood search for just-in-time scheduling of unrelated parallel machines with a common due window. *Computers & Operations Research*, *153*, Article 106159. http://dx.doi.org/10.1016/j.cor.2023.106159.

Rolim, G. A., Tomazella, C. P., & Nagano, M. S. (2024a). On the integration of reinforcement learning and simulated annealing for the parallel batch scheduling problem with setups [source code]. http://dx.doi.org/10.24433/CO.7258427.v2, URL: https://codeocean.com/capsule/5355588/tree/v2.

Rolim, G. A., Tomazella, C. P., & Nagano, M. S. (2024b). Problem instances for scheduling unrelated parallel batch machines with family setups. http://dx.doi.org/10.17632/nmfsh6ygtk.3, URL: https://data.mendeley.com/datasets/nmfsh6ygtk/3.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, *40*(4), 455–472. http://dx.doi.org/10.1287/trsc.1050.0135.

Sacramento, D., Pisinger, D., & Ropke, S. (2019). An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones. *Transportation Research Part C: Emerging Technologies*, *102*, 289–315. http://dx.doi.org/10.1016/j.trc.2019.02.018.

Sáenz-Alanís, C. A., Vallikavungal, J., Salazar-Aguilar, M. A., & Boyer, V. (2016). A parallel machine batch scheduling problem in a brewing company. *International Journal of Advanced Manufacturing Technology*, *87*(1–4), 65–75. http://dx.doi.org/10.1007/s00170-016-8477-8.

Santini, A., Ropke, S., & Hvattum, L. M. (2018). A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. *Journal of Heuristics*, *24*(5), 783–815. http://dx.doi.org/10.1007/s10732-018-9377-x.

Santos, H. G., Toffolo, T. A. M., Silva, C. L. T. F., & Vanden Berghe, G. (2016). Analysis of stochastic local search methods for the unrelated parallel machine scheduling problem. *International Transactions in Operational Research*, *26*(2), 707–724. http://dx.doi.org/10.1111/itor.12316.

Shahvari, O., Logendran, R., & Tavana, M. (2022). An efficient model-based branch-and-price algorithm for unrelated-parallel machine batching and scheduling problems. *Journal of Scheduling*, *25*(5), 589–621. http://dx.doi.org/10.1007/s10951-022-00729-7.

Silberholz, J., Golden, B., Gupta, S., & Wang, X. (2019). Computational comparison of metaheuristics. In M. Gendreau, & J. Y. Potvin (Eds.), *International series in operations research & management science*: *vol. 272*, *Handbook of metaheuristics* (pp. 581–604). Springer, http://dx.doi.org/10.1007/978-3-319-91086-4_18, chapter 18.

Smith, W. E. (1956). Various optimizers for single-stage production. *Naval Research Logistics*, *3*(1–2), 59–66.

Talbi, E.-G. (2021). Machine learning into metaheuristics: A survey and taxonomy. *ACM Computing Surveys*, *54*(6), 1–32. http://dx.doi.org/10.1145/3459664.

Tian, J., Fu, R., & Yuan, J. (2014). Online over time scheduling on parallel-batch machines: A survey. *Journal of the Operations Research Society of China*, *2*(4), 445–454. http://dx.doi.org/10.1007/s40305-014-0060-0.

Turkeš, R., Sörensen, K., & Hvattum, L. M. (2021). Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European Journal of Operational Research*, *292*(2), 423–442. http://dx.doi.org/10.1016/j.ejor.2020.10.045.

Ulaga, L., Đurasević, M., & Jakobović, D. (2022). Local search based methods for scheduling in the unrelated parallel machines environment. *Expert Systems with Applications*, *199*, Article 116909. http://dx.doi.org/10.1016/j.eswa.2022.116909.

Unlu, Y., & Mason, S. J. (2010). Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems. *Computers & Industrial Engineering*, *58*(4), 785–800. http://dx.doi.org/10.1016/j.cie.2010.02.012.

Uzsoy, R. (1995). Scheduling batch processing machines with incompatible job families. *International Journal of Production Research*, *33*(10), 2685–2708. http://dx.doi.org/10.1080/00207549508904839.

Vallada, E., & Ruiz, R. (2011). A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times. *European Journal of Operational Research*, *211*(3), 612–622. http://dx.doi.org/10.1016/j.ejor.2011.01.011.

Yang, F., Davari, M., Wei, W., Hermans, B., & Leus, R. (2022). Scheduling a single parallel-batching machine with non-identical job sizes and incompatible job families. *European Journal of Operational Research*, *303*(2), 602–615. http://dx.doi.org/10.1016/j.ejor.2022.03.027.

Zhang, H., Jia, Z. H., & Li, K. (2020). Ant colony optimization algorithm for total weighted completion time minimization on non-identical batch machines. *Computers & Operations Research*, *117*, Article 104889. http://dx.doi.org/10.1016/j.cor.2020.104889.

Zhang, H., Li, K., Jia, Z. H., & Chu, C. (2023). Minimizing total completion time on non-identical parallel batch machines with arbitrary release times using ant colony optimization. *European Journal of Operational Research*, *309*(3), 1024–1046. http://dx.doi.org/10.1016/j.ejor.2023.02.015.